

Reference Notes

Week 3: Nested While Loops

7th Grade Computer Science - Module 2

1 Introduction

1.1 This Week's Big Question

How can we break down complex problems into manageable pieces? You've mastered single while loops, but what happens when one loop isn't enough? This week, we discover how loops inside loops create powerful patterns and solve multi-dimensional problems!

Prerequisites

Before starting this week, you should be comfortable with:

- Creating and using **while** loops (Week 1)
- Using **break** and **continue** statements (Week 2)
- Writing conditions with comparison operators (Module 1, Week 7)
- Understanding logical operators: **and**, **or**, **not** (Module 1, Week 8)

1.2 What You Already Know

You can create while loops that repeat based on conditions. You know how to control loop flow with **break** and **continue**. You understand how to combine conditions using logical operators. Now we'll combine these skills to create sophisticated nested structures!

1.3 What You'll Be Able to Do

By the end of this week, you'll:

- Design programs using nested while loops for 2D patterns
- Apply logical operators to create complex loop conditions
- Recognize and implement common pattern generation algorithms
- Debug multi-level loop structures using trace tables
- Build number pyramids and geometric patterns
- Solve problems requiring multiple levels of iteration

2 VIDEO 1: Introduction to Nested While Loops

2.1 Understanding Nested Loops

A nested loop is simply a loop inside another loop. Think of it like a clock - the minute hand completes a full rotation (inner loop) for each hour the hour hand moves (outer loop). The inner loop runs completely for each iteration of the outer loop!

2.2 Your First Nested Loop

Let's create a simple nested loop to draw a rectangle of stars. The following code uses an outer loop to control the rows and an inner loop to control the columns.

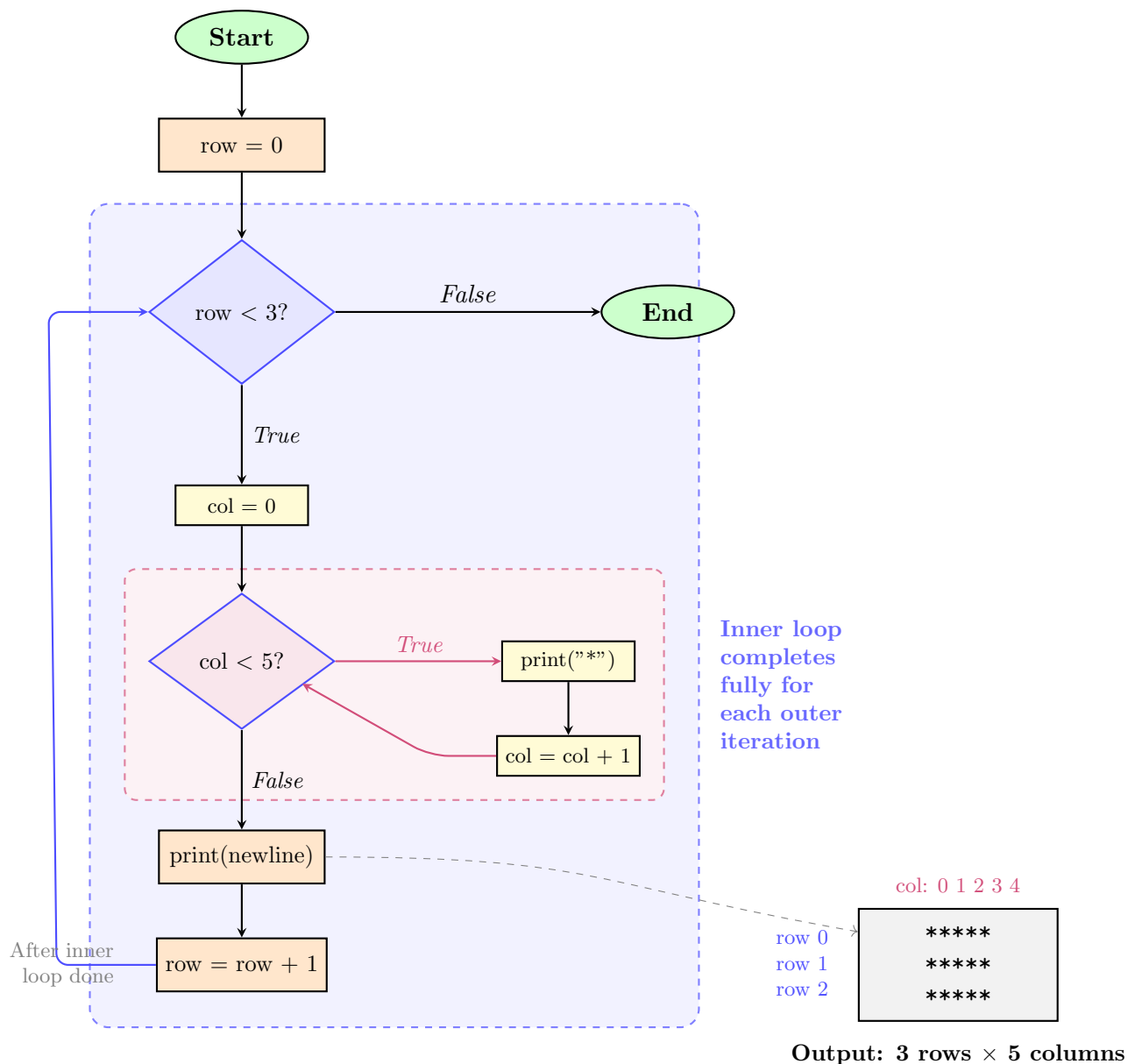
```

1 # Draw a rectangle of stars
2 row = 0
3 while row < 3:          # Outer loop: controls rows
4     col = 0
5     while col < 5:      # Inner loop: controls columns
6         print("*", end=" ")
7         col = col + 1
8     print()             # New line after each row
9     row = row + 1

```

2.3 Visualizing Nested Loop Flow

Let's visualize the flow of our nested loops using a flowchart. This will help you understand how the outer and inner loops interact.



2.4 How the Loops Work Together

Let's see how the outer and inner loops work together in practice.

```

1 # Track execution with print statements
2 outer = 1
3 while outer <= 2:
4     print(f"Outer loop: {outer}")
5     inner = 1
6     while inner <= 3:
7         print(f"    Inner loop: {inner}")
8         inner = inner + 1
9     outer = outer + 1
10 # Output shows inner runs 3 times for each outer
  
```

💡 Rule

Nested Loop Golden Rule: The inner loop runs COMPLETELY for EACH iteration of the outer loop. If outer runs 3 times and inner runs 4 times, total iterations = $3 \times 4 = 12$

⚠️ Common Error

Common Mistake: Forgetting to reset the inner loop variable
Wrong:

```
1 col = 0 # Only initialized once!
2 row = 0
3 while row < 3:
4     while col < 5: # Only works first time!
5         print("*", end=" ")
6         col = col + 1
7     row = row + 1
```

Right: Reset `col = 0` inside the outer loop, before the inner loop.

👉 Quick Check

How many times will "Hello" be printed?

```
1 x = 0
2 while x < 2:
3     y = 0
4     while y < 3:
5         print("Hello")
6         y = y + 1
7     x = x + 1
```

3 VIDEO 2: Complex Conditions with Logical Operators

3.1 Understanding Complex Conditions

While loops become more powerful when we combine multiple conditions using logical operators. This lets us create sophisticated control flow that responds to multiple factors simultaneously.

3.2 Using AND in Loop Conditions

We can use 'and' to require multiple conditions to be true for the loop to continue.

```
1 # Continue only if BOTH conditions are true
2 x = 0
3 y = 0
4 while x < 5 and y < 3:
5     print(f"x={x}, y={y}")
6     x = x + 1
7     y = y + 1
8 # Stops when EITHER condition becomes false
```

3.3 Using OR in Loop Conditions

We can also use 'or' to continue the loop if at least one condition is true.

```

1 # Continue if EITHER condition is true
2 attempts = 0
3 correct = False
4 while attempts < 3 or not correct:
5     password = input("Enter password: ")
6     if password == "secret":
7         correct = True
8         break
9     attempts = attempts + 1

```

3.4 Comparing Logical Operators

Operator	Meaning	Loop Continues When	Loop Stops When
and	Both true	ALL conditions true	ANY condition false
or	Either true	ANY condition true	ALL conditions false
not	Opposite	Condition is false	Condition becomes true

3.5 Complex Conditions in Nested Loops

Sometimes we need complex conditions in nested loops. For example, controlling a game character's movement within a grid requires checking multiple boundaries.

```

1 # Game grid: stay within bounds
2 x, y = 5, 5 # Start position
3 while True:
4     move = input("Move (w/a/s/d): ")
5     # Complex condition checks all boundaries
6     if move == "w" and y > 0:
7         y = y - 1
8     elif move == "s" and y < 9:
9         y = y + 1
10    elif move == "a" and x > 0:
11        x = x - 1
12    elif move == "d" and x < 9:
13        x = x + 1
14    print(f"Position: ({x}, {y})")

```

Tip

Key Observations:

- The **and** operator ensures BOTH conditions must be true: the correct key AND valid boundary
- When at boundary (x=0 or y=0 or x=9 or y=9), moves that would go outside are blocked
- Position stays the same when a move is blocked
- Valid grid range: x from 0 to 9, y from 0 to 9 (10×10 grid)

Tip

Debugging Complex Conditions: Break them down! Instead of one long condition, use variables:

```
1 in_bounds = x >= 0 and x < 10 and y >= 0 and y < 10
2 has_energy = energy > 0
3 can_move = in_bounds and has_energy
4 while can_move:
5     # Easier to debug!
```

Quick Check

When will this loop stop?

```
1 a = 0
2 b = 10
3 while a < 5 or b > 0:
4     a = a + 1
5     b = b - 3
```

Hint: Track both variables and check when BOTH parts of the OR become false.

4 VIDEO 3: Pattern Generation with Nested Loops

4.1 Understanding Pattern Building

Nested loops excel at creating 2D patterns. The outer loop typically controls rows (vertical), while the inner loop controls what happens in each row (horizontal). By varying the inner loop's behavior based on the outer loop's counter, we create dynamic patterns!

4.2 Creating a Number Triangle

```
1 # Right triangle pattern
2 row = 1
3 while row <= 3:
4     col = 1
5     while col <= row: # Inner depends on outer!
6         print(col, end=" ")
7         col = col + 1
8     print()
9     row = row + 1
```

4.3 Visualizing Pattern Generation

The following trace table shows how the inner loop's limit changes with each outer loop iteration, creating a triangle pattern.

Step	Loop	row	col	Condition Check	Output
1	Outer	1	—	$\text{row} \leq 3? \rightarrow 1 \leq 3 \checkmark$	—
2	Inner	1	1	$\text{col} \leq \text{row}? \rightarrow 1 \leq 1 \checkmark$	1
3	Inner	1	2	$\text{col} \leq \text{row}? \rightarrow 2 \leq 1 \times$	—
4	Outer	1	—	Print newline, row = 2	(newline)
5	Outer	2	—	$\text{row} \leq 3? \rightarrow 2 \leq 3 \checkmark$	—
6	Inner	2	1	$\text{col} \leq \text{row}? \rightarrow 1 \leq 2 \checkmark$	1
7	Inner	2	2	$\text{col} \leq \text{row}? \rightarrow 2 \leq 2 \checkmark$	2
8	Inner	2	3	$\text{col} \leq \text{row}? \rightarrow 3 \leq 2 \times$	—
9	Outer	2	—	Print newline, row = 3	(newline)
10	Outer	3	—	$\text{row} \leq 3? \rightarrow 3 \leq 3 \checkmark$	—
11	Inner	3	1	$\text{col} \leq \text{row}? \rightarrow 1 \leq 3 \checkmark$	1
12	Inner	3	2	$\text{col} \leq \text{row}? \rightarrow 2 \leq 3 \checkmark$	2
13	Inner	3	3	$\text{col} \leq \text{row}? \rightarrow 3 \leq 3 \checkmark$	3
14	Inner	3	4	$\text{col} \leq \text{row}? \rightarrow 4 \leq 3 \times$	—
15	Outer	3	—	Print newline, row = 4	(newline)
16	Outer	4	—	$\text{row} \leq 3? \rightarrow 4 \leq 3 \times$	Done!

Tip

Final Output:

```
1
1 2
1 2 3
```

Loop Statistics:

- **Outer loop:** 3 complete iterations (row = 1, 2, 3)
- **Inner loop:** $1 + 2 + 3 = 6$ total print statements
- Notice how col resets to 1 for each new row!

4.4 Creating a Multiplication Table

Using similar logic, we can easily create a multiplication table as follows:

```
1 # Multiplication table (partial)
2 row = 1
3 while row <= 5:
4     col = 1
5     while col <= 5:
6         product = row * col
7         print(f"{product:3}", end=" ")
8         col = col + 1
9     print()
10    row = row + 1
```

The outer loop controls the rows (1 to 5), while the inner loop controls the columns (1 to 5). Each cell prints the product of the current row and column. This gives us the final output:

```
1  2  3  4  5
2  4  6  8 10
3  6  9 12 15
4  8 12 16 20
5 10 15 20 25
```

4.5 Common Pattern Techniques

Pattern Type	Inner Loop Technique	Example Output
Right triangle	col <= row	*, **, ***
Inverted triangle	col <= (n - row)	***, **, *
Square	col <= n	Fixed width
Diamond	Combine ascending + descending	/\ shape

The following code creates an inverted triangle pattern:

```
1 row = 0
2 size = 4
3 while row < size:
4     col = 0
5     while col < (size - row):
6         print("*", end=" ")
7         col = col + 1
8     print()
9     row = row + 1
```

Output:

```
* * * *
* * *
* *
*
```

⚠ Common Error

Common Mistake: Using the same variable name for inner and outer loops

```
1 i = 0
2 while i < 3:
3     i = 0 # Oops! This resets outer loop variable!
4     while i < 3:
5         print("*")
6         i = i + 1 # This affects outer loop!
```

Always use different variable names: row/col, i/j, outer/inner

Quick Check

What pattern will this create?

```

1 line = 1
2 while line <= 3:
3     space = 1
4     while space <= (3 - line):
5         print(" ", end="")
6         space = space + 1
7     star = 1
8     while star <= line:
9         print("*", end="")
10        star = star + 1
11    print()
12    line = line + 1
    
```

5 VIDEO 4: Debugging Nested Loops

5.1 Understanding Nested Loop Complexity

Debugging nested loops requires tracking multiple variables simultaneously. The key is to trace execution systematically, understanding exactly when each loop starts, iterates, and ends.

5.2 Using Print Statements Strategically

Strategically placing print statements can help illuminate the flow of nested loops.

```

1 # Add debug prints at key points
2 outer = 0
3 while outer < 2:
4     print(f"OUTER START: outer={outer}")
5     inner = 0
6     while inner < 3:
7         print(f"  INNER: outer={outer}, inner={inner}")
8         inner = inner + 1
9     print(f"OUTER END: outer={outer}")
10    outer = outer + 1
    
```

5.3 Understanding Nested Loop Execution (*Trace Table*)

Let's trace through this simple nested loop:

```

1 # Very simple nested loop
2 outer = 0
3 while outer < 2:
4     inner = 0
5     while inner < 2:
6         print(f"({outer},{inner})", end=" ")
7         inner = inner + 1
8     print() # New line
9     outer = outer + 1
    
```

Step	Loop	outer	inner	Condition Check	Output
1	Outer	0	—	outer < 2? → 0 < 2 ✓	—
2	Inner	0	0	inner < 2? → 0 < 2 ✓	(0,0)
3	Inner	0	1	inner < 2? → 1 < 2 ✓	(0,1)
4	Inner	0	2	inner < 2? → 2 < 2 ✗	—
5	Outer	1	reset	outer = 1, check outer < 2? ✓	(newline)
6	Inner	1	0	inner < 2? → 0 < 2 ✓	(1,0)
7	Inner	1	1	inner < 2? → 1 < 2 ✓	(1,1)
8	Inner	1	2	inner < 2? → 2 < 2 ✗	—
9	Outer	2	—	outer < 2? → 2 < 2 ✗	(newline)
Total Steps: 9		Final Output: (0,0) (0,1) on line 1, (1,0) (1,1) on line 2			

What You See on Screen:

```
(0,0) (0,1)
(1,0) (1,1)
```

Key Points:

- Blue rows = Outer loop (runs 2 times)
- Purple rows = Inner loop (runs 2 times for EACH outer)
- Yellow highlight = Inner variable resets to 0
- Total iterations: 2 outer × 2 inner = 4 prints

5.4 Common Nested Loop Bugs

Bug 1: Infinite Inner Loop

This happens when the inner loop variable is not updated or reset properly, resulting in an infinite loop:

```
1 # Inner variable not updating correctly
2 row = 0
3 while row < 3:
4     col = 0
5     while col < 5:
6         print("*", end=" ")
7         # Forgot: col = col + 1 <- Missing!
8     row = row + 1
```

Bug 2: Wrong Variable in Condition

This is also a very common mistake, where the inner loop condition mistakenly uses the outer loop variable:

```
1 # Using wrong variable in inner condition
2 i = 0
3 while i < 3:
4     j = 0
```

```

5 while i < 5: # Should be j < 5!
6     print(" ", end=" ")
7     j = j + 1
8     i = i + 1

```

5.5 Debugging with Break Points

One effective strategy is to use **break** statements to limit iterations while debugging. This helps isolate which loop is causing issues.

```

1 # Use break to isolate problems
2 outer = 0
3 debug_mode = True
4 while outer < 10:
5     inner = 0
6     while inner < 10:
7         if debug_mode and inner > 2:
8             print("DEBUG: Breaking inner loop")
9             break # Test with limited iterations
10            # Your complex code here
11            inner = inner + 1
12        if debug_mode and outer > 1:
13            break # Test with limited outer iterations
14        outer = outer + 1

```

Tip

Nested Loop Debugging Strategy:

1. First, test outer loop alone (comment out inner loop)
2. Then test inner loop with fixed outer value
3. Finally, test both together with small limits
4. Use descriptive variable names (row/col not i/j)

Quick Check

Find the bug: Why does this create an infinite loop?

```

1 x = 0
2 while x < 3:
3     y = 0
4     while y < x: # Hint: What happens when x = 0?
5         print(" ")
6         y = y + 1
7     # Missing something here?

```

6 VIDEO 5: Summary

6.1 What We Learned This Week

This week, you conquered nested while loops and complex conditions! You can now create 2D patterns, build multiplication tables, and control sophisticated program flow with logical operators. You've

learned that nested loops multiply complexity but also multiply power - enabling you to solve multi-dimensional problems that single loops can't handle.

6.2 Key Concepts We Covered

Concept	Key Points	Example Use
Nested loops	Inner runs completely per outer	Grid/pattern generation
Loop coordination	Inner often depends on outer	Triangle patterns
Complex conditions	Combine with and/or/not	Multi-factor control
Pattern algorithms	Row/column structure	Number pyramids
Debug strategies	Trace tables, print statements	Isolate loop levels
Variable scope	Reset inner variables	Avoid infinite loops

6.3 Patterns You Can Now Create

- **Geometric shapes:** Triangles, rectangles, diamonds
- **Number patterns:** Pyramids, multiplication tables
- **Game grids:** 2D game boards, coordinate systems
- **Data tables:** Formatted output in rows and columns

★ Landmark Moment

You've unlocked the power of multi-dimensional thinking! With nested loops, you can now process data in 2D grids, create complex patterns, and build the foundation for games and simulations. You're thinking like a programmer who can handle complexity by breaking it into manageable, nested pieces!

7 Quick Reference

Quick Reference

Week 3 Essential Patterns:

Basic Nested Loop:

- Pattern: `while outer: while inner:`
- Common use: 2D patterns, grids
- Common mistake: Forgetting to reset inner variable

Triangle Pattern:

- Pattern: `while col <= row:`
- Common use: Number pyramids
- Debug tip: Print row and col values

Complex Conditions:

- Pattern: `while (cond1 and cond2) or cond3:`
- Common use: Multi-factor decisions
- Debug tip: Break into separate boolean variables

Grid Navigation:

- Pattern: `0 <= x < width and 0 <= y < height`
- Common use: Game boards, coordinate systems
- Common mistake: Off-by-one boundary errors

7.1 Reflection Questions

1. Draw a trace table for a nested loop you wrote this week. How many total iterations occurred? Did it match your prediction?
2. When creating patterns, how do you decide what the inner loop's condition should be?
3. Think of a real-world situation that's like nested loops (hint: calendars, schedules, sports tournaments). How do the "loops" work together?
4. What's the most complex condition you wrote this week? Could you simplify it using helper variables?
5. Look at a checkerboard or chess board. How would you use nested loops to represent it in a program?

Next week: We dive into strings - discovering how Python stores and manipulates text, and why strings behave differently from numbers!